Module

1

Introduction to Finite Automata

Introduction to Finite Automata introduces students to the foundational concepts of automata theory, focusing on deterministic and non-deterministic finite automata. This module builds intuition around how machines recognize patterns and form the basis of computation models.

1.1

Formal Languages and Automata Theory | Course Overview

This session introduces the importance of Automata Theory for CS students, its connection to real-world systems, and areas of application. Learners also explore how to effectively study the course and gain an overview of the modules ahead.

1.2

Optional

Formal Languages and Automata Theory | Pre-requisites for this Course

Formal Languages and Automata Theory | Pre-requisites for this Course introduces the foundational mathematical concepts required for understanding automata theory. This LU covers sets, relations, functions, logic, and proof techniques, ensuring students are equipped to engage with formal models of computation.

1.3

Introduction to Automata Theory | Live Session

Dive into the foundations of theoretical computer science and explore how abstract machines like automata model computation. This session motivates the study of pattern-recognizing machines and sets the roadmap for the course.

1.4

Finite State Machine and Regular Languages

Learn what finite state machines are and how they define regular languages. Discover how simple machines recognize structured patterns—a key idea behind programming languages, compilers, and text processing.

1.5

Deterministic Finite Automata

Understand the structure and behavior of Deterministic Finite Automata (DFA), essential machines for recognizing regular languages, providing a clear and precise way to design pattern recognizers.

1.6

Optional

Practice Assignment | Deterministic Finite Automata

This is a practice assignment for the previous lesson

1.7

Assignment | Construction of DFAs

Apply your knowledge of DFAs by constructing machines for different languages. This hands-on session sharpens your ability to interpret rules and design deterministic models for structured input.

1.8

Non-deterministic Finite Automata and Epsilon NFA

Explore non-deterministic finite automata (NFA) and epsilon-NFA. Understand how ε-transitions work, how NFAs differ from DFAs, and how they simplify the construction of automata without changing the recognized language.

1.9

Assignment | NFA to DFA conversion

Practice converting an NFA into a DFA using the subset construction algorithm. Mastering this skill is crucial for understanding formal languages and building efficient automata-based models.

Module

2

Regular Expression

Regular Expression explores the formal language used to describe patterns in strings, and its equivalence to finite automata. This module equips with the theoretical foundation and practical skills to design, interpret, and convert regular expressions.

2.1

Introduction to Regular Expressions | Live Session

Dive into the world of regular expressions and see how symbolic notations form the backbone of pattern recognition in computing. This session sets the stage for defining, processing, and verifying strings precisely.

2.2

Kleene's theorem and Operations on Regular Expressions

Understand the fundamental bridge between regular expressions and finite automata through Kleene's Theorem. Discover how operations like union, concatenation, and closure extend pattern expressiveness.

2.3

Optional

Practice Assignment | Kleene's theorem and Operations on Regular Expressions

This is a practice assignment for the previous lesson

2.4

Finite Automata to Regular Expression conversion

Learn how to reverse engineer automata back into regular expressions. This transformation deepens understanding of the equivalence between regular expressions and finite automata.

2.5

State Elimination Method for Finite Automata to Regular Expression

Explore the structured process of reducing a finite automaton to a regular expression using the state elimination method. Each step reveals the pattern the automaton recognizes.

2.6

Optional

Assignment | Finite Automata to Regular Expression

Practice the method of translating finite automata into regular expressions through detailed exercises. Build fluency with both symbolic representation and automation logic.

2.7

Regular Expression to Finite Automata conversion

Discover how to construct a finite automaton that accepts a given regular expression. This transformation translates symbolic language rules into machine behavior.

2.8

Optional

Assignment | Regular Expression to Finite Automata

This assignment provides practice in converting regular expressions into corresponding finite automata. It reinforces construction steps and solidifies theoretical knowledge through examples.

2.9

Pumping Lemma for Regular Languages

Learn how to prove a language is not regular using the Pumping Lemma. Develop formal proof techniques and understand the limitations of regular expressions and automata.

2.10

Chomsky Hierarchy

Learn about the classification of formal languages based on complexity and the machines that recognize them. The Chomsky hierarchy structures understanding of language models.

2.11

Non-Regular Languages and Regular Grammar

Explore two concepts: identifying non-regular languages and understanding regular grammars. Learn formal definitions, study right-linear/left-linear grammars, and see how grammars relate to finite automata.

2.12

Assignment | Minimisation of DFA

Learn how to minimize a DFA by identifying and merging equivalent states. This reduces the machine's size while preserving its functionality—key in optimization and practical use.

Module

3

Context Free Grammar and Languages

Context-Free Grammar and Languages explores the structure of formal languages through context-free grammars, which form the backbone of programming language syntax. This module introduces parse trees, derivations, and the classification of languages beyond regular expressions.

3.1

Context Free Grammar(CFG) and Context Free Language(CFL) | Live Session

This session introduces context-free grammars, which define most programming languages. Learn how CFGs describe context-free languages and prepare for deeper exploration into parsing and syntax checking.

3.2

Derivation Trees and Ambiguous Grammar

Explore how CFGs generate strings through derivations and parse trees. Learn about grammar ambiguity, its problems in compilers, and how it affects parsing.

3.3

Ambiguous to Unambiguous Context Free Grammar

Learn to identify ambiguous grammars and convert them into unambiguous ones. This skill is critical for designing reliable parsers in compiler construction.

3.4

Simplification of Context Free Grammar | Live Session

Focus on simplifying grammars by eliminating useless symbols, ε-productions, and unit rules while preserving language structure, improving efficiency and readability.

3.5

Chomsky Normal Form(CNF) and Greibach Normal Form (GNF)

Understand CNF and GNF, two standard forms of CFGs useful for parsing and theoretical analysis. Learn their formats and importance in grammar processing.

3.6

CFG to CNF conversion | Live Session

Practice converting CFGs into Chomsky Normal Form through guided examples. This live session builds step-by-step understanding of CNF transformations.

3.7

Optional

[OPTIONAL] CYK algorithm

CYK Algorithm introduces the Cocke-Younger-Kasami algorithm used to determine whether a string belongs to a context-free language. This module emphasizes bottom-up parsing using dynamic programming and applies the technique to grammars in Chomsky Normal Form.

3.8

CFG to GNF conversion | Live Session

Learn how to convert any CFG into Greibach Normal Form (GNF). Gain confidence in left-recursion elimination and left-factoring—key steps in predictive parsing.

3.9

Optional

Assignment | CFG to CNF conversion and CFG to GNF conversion

This assignment reinforces grammar normalization. Practice converting CFGs to CNF and GNF, ensuring language preservation through formal transformations.

3.10

Pumping lemma for Context Free Languages

Learn the pumping lemma for CFLs to prove languages are not context-free. Understand the decomposition conditions and build proofs by contradiction.

3.11

Optional

Closure and Decision properties of Context Free Languages

Explore which operations preserve context-free properties and which decision problems are solvable for CFLs, strengthening your algorithmic reasoning.

3.12

Assignment | Ambiguous Grammar and Simplification of Grammar

In this assignment, apply your knowledge of grammar ambiguity by rewriting grammars into simpler, unambiguous forms and reducing grammar size efficiently.